```
     function changeState(Phase x) onlyChair public {              ❷

        require (x > state );                                      ❸
        state = x;
     }

    function register(address voter) public validPhase(Phase.Regs)
                                                  onlyChair {      ❹

        require (! voters[voter].voted);                           ❸

        voters[voter].weight = 1;
       // voters[voter].voted = false;
    }


    function vote(uint toProposal) public validPhase(Phase.Vote)  {

        Voter memory sender = voters[msg.sender];                  ❺

        require (!sender.voted);                                   ❻
        require (toProposal < proposals.length);                   ❻

        sender.voted = true;
        sender.vote = toProposal;
        proposals[toProposal].voteCount += sender.weight;
    }

    function reqWinner() public validPhase(Phase.Done) view
                            returns (uint winningProposal)
    {

        uint winningVoteCount = 0;
        for (uint prop = 0; prop < proposals.length; prop++)
            if (proposals[prop].voteCount > winningVoteCount) {
                winningVoteCount = proposals[prop].voteCount;
                winningProposal = prop;
            }
       assert(winningVoteCount>=3);                                ❼
    }
}
```

❶ Two modifiers, including onlyChair

❷ Use of onlyChair modifier

❸ require() instead of traditional if

❹ Use of two modifiers: validPhase and onlyChair

❺ Use of memory instead of storage type for local variables

❻ require() instead of traditional if

❼ Use of assert()